UD02: MONITORIZACIÓN Y ALTA DISPONIBILIDAD EN MÁQUINAS VIRTUALES

ACTIVIDAD 2.1: ALARMAS DE CLOUDWATCH: AVISOS Y RECUPERACIÓN AUTOMÁTICA

En esta actividad aprenderemos a **vigilar una instancia EC2** con **Amazon CloudWatch** y a conseguir que **reaccione sola** ante los fallos. Consultaremos sus **métricas** desde la consola y desde la CLI, crearemos **alarmas** que avisan por correo con **Amazon SNS** y alarmas que **detienen, recuperan o reinician** la instancia, y lo reuniremos todo en un **panel**.

Provocaremos cuatro situaciones: un fallo del **host físico** de AWS (simulado, porque no lo podemos romper), un pico de **CPU** generado con stress-ng, un fallo **dentro del sistema operativo**, desactivando su interfaz de red, y la caída del **servidor web**, que las comprobaciones de la máquina no detectan. En cada caso veremos qué comprobación lo detecta, cómo cambia el estado de la alarma y qué acción se ejecuta.

La actividad crea todo lo que necesita desde cero. Al terminar **no se borra**: la instancia, el tema de SNS, las alarmas y el panel se reutilizan en la siguiente actividad. La infraestructura se crea con la **consola**; la CLI se usa para consultar métricas y alarmas y para forzar el estado de una alarma. Algunos comandos no se dan completos: tendrás que construirlos con las pistas y con la ayuda de la CLI (aws <servicio> <operación> help).

Tipo: actividad guiada por el profesor/a, sin entregable. **Duración estimada:** dos sesiones.

> **Nota:** los comandos están escritos para **bash** y la **AWS CLI v2**. Puedes ejecutarlos en **CloudShell**, que ya tiene las credenciales del laboratorio y es cómodo para pocos comandos, o en el **terminal del ordenador de clase**, después de copiar las credenciales del laboratorio como de costumbre. La interfaz de AWS Academy y de la consola de AWS cambia con frecuencia, por lo que algunos menús o la salida de algunos comandos pueden variar ligeramente, pero los conceptos son los mismos.

OBJETIVOS

- Consultar las métricas de una instancia en la consola y con la CLI, y distinguir la monitorización básica de la detallada.
- Diferenciar las comprobaciones de estado del sistema y de la instancia, y elegir la acción adecuada para cada una.
- Configurar alarmas con umbral, periodo, M de N y tratamiento de datos ausentes, y seguir sus cambios de estado.
- Enviar avisos por correo electrónico con un tema de SNS.

- Configurar una comprobación del estado de la aplicación y distinguirla de las comprobaciones del sistema y de la instancia.
- Construir un panel con métricas y con el estado de las alarmas.

PARTE 1: PREPARACIÓN

Inicia el laboratorio y lanza desde la consola de EC2 una instancia con estos datos. Sustituye **xx** por tus iniciales en todos los nombres de la práctica.

- **Nombre y etiquetas:** Name = mon-xx. Con **Agregar etiquetas adicionales**, añade Proyecto = UD02 y Propietario = xx, aplicadas a instancias y volúmenes.
- **AMI:** Ubuntu Server 26.04 LTS (64 bits, x86). **Tipo:** t3.micro.
- **Par de claves:** vockey. Entraremos por **SSH** con la clave del laboratorio (labsuser.pem).
- **Configuración de red:** crea el grupo de seguridad mon-xx-sg con dos reglas de entrada, **SSH** y **HTTP**, ambas desde **213.0.86.0/23** (el rango del instituto) o desde el que indique el profesor/a. Si trabajas desde casa, puedes usar «Mi IP». No es recomendable abrir SSH a 0.0.0.0/0 fuera de un entorno de pruebas.
- **Detalles avanzados:** perfil de instancia **LabInstanceProfile** (permite entrar también con Session Manager si lo necesitas) y, en **Datos de usuario**, el script siguiente.

```
#!/bin/bash
apt-get update
apt-get install -y stress-ng apache2
```

El script instala **stress-ng**, un generador de carga que usaremos para disparar alarmas, y el servidor web **Apache**. Mientras la instancia arranca, abre el terminal en el que vayas a usar la CLI y guarda su ID en una variable:

```
# AWS CLI
OWNER=xx
ID=$(aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=mon-$OWNER" \
  "Name=instance-state-name,Values=pending,running" \
  --query "Reservations[0].Instances[0].InstanceId" --output text)
echo $ID
```

Cuando la instancia esté en marcha, espera un par de minutos a que terminen los datos de usuario y abre en el navegador http://<IP pública de mon-xx>. Debe aparecer la página por defecto de Apache.

Las variables se pierden si cierras el terminal o si CloudShell se cierra por inactividad. En ese caso, vuelve a ejecutar este bloque.

Los bloques **azul claro** se ejecutan en la CLI (CloudShell o tu equipo); los **amarillos**, en la instancia, y los **grises** son scripts que se pegan en la consola.

PARTE 2: MÉTRICAS DE EC2

EC2 envía a CloudWatch, sin instalar nada, métricas que mide **desde fuera** de la instancia: CPU, tráfico de red, créditos de CPU y comprobaciones de estado, entre otras. Se guardan en el **espacio de nombres** AWS/EC2 y se distinguen por la **dimensión** InstanceId.

En la consola, selecciona mon-xx y abre la pestaña **Monitorización**. Después ve a **CloudWatch > Métricas > Todas las métricas > EC2 > Métricas por instancia**, busca tu ID y marca CPUUtilization. En la pestaña **Métricas diagramadas** prueba a cambiar la estadística y el periodo.

Desde la CLI, list-metrics dice qué métricas existen y get-metric-statistics devuelve sus datos. Si la instancia acaba de arrancar, espera unos minutos a que lleguen los primeros. Este es el modelo para el resto de la parte:

```
# AWS CLI
aws cloudwatch list-metrics --namespace AWS/EC2 \
  --dimensions Name=InstanceId,Value=$ID \
  --query "Metrics[].MetricName" --output text | tr '\t' '\n' | sort

INI=$(date -u -d '-1 hour' +%FT%TZ)
FIN=$(date -u +%FT%TZ)
aws cloudwatch get-metric-statistics --namespace AWS/EC2 \
  --metric-name CPUUtilization --dimensions Name=InstanceId,Value=$ID \
  --start-time $INI --end-time $FIN --period 300 \
  --statistics Average Maximum --output table \
  --query "sort_by(Datapoints,&Timestamp)[].{Hora:Timestamp,
  Media:Average,Max:Maximum}"
```

Ejercicios. Construye los comandos a partir del modelo. Comprueba cada resultado en la consola.

- Repite la consulta de CPU con --period 60. ¿Obtienes más datos? Con la **monitorización básica**, EC2 envía un dato cada **5 minutos**.
- Consulta StatusCheckFailed_System con la estadística Maximum y un periodo de 60 segundos. Las comprobaciones de estado se publican **cada minuto**, incluso con la monitorización básica. ¿Qué valor tienen ahora?
- Activa la **monitorización detallada** de mon-xx (**Acciones > Supervisar y solucionar problemas > Administrar la monitorización detallada**, o aws ec2 monitor-instances). Repite la consulta de CPU con --period 60 dentro de unos diez minutos: debe haber un dato por minuto.

> **Para pensar:** la monitorización detallada tiene coste. ¿Cuándo compensa? Pista: calcula cuánto tarda en saltar una alarma que necesita tres datos seguidos por encima del umbral si llega un dato cada 5 minutos.
>
> **Para pensar:** entre las métricas no aparecen ni la memoria ni el espacio libre en disco. ¿Por qué EC2 no puede medirlos desde fuera de la instancia?

PARTE 3: RECUPERACIÓN ANTE UN FALLO DEL SISTEMA

EC2 hace cada minuto dos **comprobaciones de estado** y publica el resultado en CloudWatch (0 si es correcto y 1 si falla):

- **Del sistema** (StatusCheckFailed_System): problemas de la infraestructura de AWS, como el host físico, la alimentación o la red. No los puedes arreglar tú: la instancia tiene que pasar a otro host.
- **De la instancia** (StatusCheckFailed_Instance): problemas dentro de la instancia, como una red mal configurada, la memoria agotada o un kernel o sistema de ficheros dañado. Normalmente se resuelven reiniciando o corrigiendo la configuración.

Existe una tercera, la de los **volúmenes EBS adjuntos** (StatusCheckFailed_AttachedEBS), que detecta volúmenes inaccesibles. Más adelante añadiremos una cuarta, la de la **aplicación**.

Recuperación automática simplificada. Ante un fallo del sistema, EC2 recupera por defecto las instancias compatibles, como las t3: las arranca en otro host y conserva el ID, las IP privadas, las IP elásticas, los volúmenes EBS y los metadatos. Compruébalo: selecciona mon-xx y ve a **Acciones > Configuración de la instancia > Cambiar el comportamiento de recuperación automática**. Debe estar en **Predeterminado**, que significa activada. No cambies nada.

Con una **alarma de CloudWatch** sobre StatusCheckFailed_System se gana control: decides cuándo se dispara, qué acción ejecuta y a quién avisa. El proceso sería este:

1. Falla la infraestructura que hay debajo de la instancia.
2. EC2 detecta el fallo y publica un 1 en la métrica StatusCheckFailed_System.
3. La alarma pasa a **En alarma** y ejecuta su acción.
4. Con la acción **Recuperar**, la instancia arranca en otro host físico.
5. Los volúmenes EBS, las IP privadas y la IP elástica (si la hay) siguen siendo los mismos.

Ejercicio. Crea una alarma que **detenga** la instancia ante un fallo del sistema. Usamos Detener y no Recuperar porque su efecto se ve en segundos y no necesita un fallo real del host; más

adelante la dejaremos como en producción. Selecciona mon-xx y ve a **Acciones > Supervisar y solucionar problemas > Administrar alarmas de CloudWatch**, opción **Crear una alarma**:

- **Notificación de alarma:** desactivada, de momento.
- **Acción de alarma:** activada, con la acción **Detener**.
- **Agrupar muestras por:** Máximo. **Tipo de datos para la muestra:** comprobación de estado del sistema fallida.
- **Periodos consecutivos:** 2. **Periodo:** 1 minuto.
- **Nombre de la alarma:** mon-xx-sistema.

Durante unos minutos la alarma estará en **Datos insuficientes**; después pasará a **OK**. Síguela en la pestaña **Estado y alarmas** de la instancia o en **CloudWatch > Alarmas > Todas las alarmas**.

Simula el fallo. No podemos romper un host de AWS, pero sí forzar el estado de la alarma:

```
# AWS CLI
aws cloudwatch set-alarm-state --alarm-name mon-$OWNER-sistema \
  --state-value ALARM --state-reason "Simulacro de fallo del host"
```

La alarma pasa a **En alarma** y la instancia empieza a detenerse. Abre la alarma y revisa la pestaña **Historial**: verás el cambio de estado y la acción ejecutada.

> **Para pensar:** set-alarm-state solo mantiene el estado hasta la siguiente evaluación. ¿En qué estado estará la alarma unos minutos después, con la instancia detenida? ¿Por qué no vuelve a OK? Pista: una instancia detenida no publica comprobaciones de estado.

PARTE 4: AVISOS POR CORREO CON SNS

Una alarma que actúa sin avisar a nadie es peligrosa: alguien tiene que saber que la instancia se ha recuperado o reiniciado. CloudWatch no envía correos: publica un mensaje en un **tema de SNS**, y el tema lo reparte entre sus **suscriptores** (correo, SMS, colas, funciones Lambda, etc.). SNS se estudiará con detalle en un tema posterior, pero en este tema lo usaremos para recibir los avisos de las alarmas.

Vuelve a iniciar mon-xx, que quedó detenida por la alarma. Si has empezado una sesión nueva del laboratorio, puede que ya esté en marcha, porque al iniciar una sesión el Learner Lab vuelve a arrancar las instancias detenidas. Su IP pública habrá cambiado. Si es necesario, vuelve a definir las variables de la parte 1. Después, en la consola de **SNS**:

1. **Temas > Crear tema**, de tipo **Estándar** y con el nombre avisos-xx.
2. Dentro del tema, **Crear suscripción** con el protocolo **Correo electrónico** y tu dirección.

3. Abre el correo de *AWS Notifications* y pulsa **Confirm subscription** (revisa la carpeta de spam). Hasta entonces, la suscripción está **pendiente de confirmación** y no recibirás nada.

Ejercicio. Envía un mensaje de prueba al tema desde la CLI con aws sns publish. Necesitarás --topic-arn, --subject y --message. El ARN del tema se obtiene así:

```
# AWS CLI
TOPIC=$(aws sns list-topics --output text \
  --query "Topics[?ends_with(TopicArn,':avisos-$OWNER')].TopicArn")
echo $TOPIC
```

Ejercicio. Deja mon-xx-sistema como estaría en producción. En CloudWatch, selecciona la alarma y pulsa **Acciones > Editar**. En el paso de las acciones, quita la acción de EC2 **Detener**, añade la acción **Recuperar esta instancia** y una **notificación** a avisos-xx cuando la alarma esté en alarma.

> **Para pensar:** si la recuperación automática simplificada ya está activada, ¿qué aporta esta alarma?

PARTE 5: ALARMA DE CPU

Ahora crearemos una alarma con el asistente de CloudWatch, que muestra todas las opciones. Queremos un aviso cuando la CPU media supere el **70 %** en **3 de los últimos 5 minutos**. Antes, repite la consulta de CPU de la parte 2 con --period 60 para confirmar que la monitorización detallada funciona.

Ve a **CloudWatch > Alarmas > Todas las alarmas > Crear alarma > Seleccionar métrica**:

1. **EC2 > Métricas por instancia** y CPUUtilization de tu instancia.
2. **Estadística:** Promedio. **Periodo:** 1 minuto.
3. **Condiciones:** Estático, **Mayor que** 70.
4. **Configuración adicional:** 3 de 5 en **Puntos de datos para la alarma** y, en **Tratamiento de datos ausentes**, la opción por defecto: tratarlos como ausentes.
5. **Acciones:** notificación a avisos-xx cuando esté **en alarma**. Añade otra notificación al mismo tema para cuando vuelva a **OK**.
6. **Nombre:** mon-xx-cpu. **Descripción:** CPU de mon-xx por encima del 70 %.

Fíjate en que el gráfico del asistente marca el umbral con una línea roja.

Genera carga. Conéctate a mon-xx por SSH (usuario ubuntu) y lanza stress-ng sobre todas las CPU durante 8 minutos:

```
# mon-xx
stress-ng --cpu 0 --timeout 8m --metrics-brief
```

--cpu 0 crea un proceso de carga por cada vCPU. Mientras se ejecuta, sigue la alarma en otra pestaña. Recibirás un correo cuando pase a **En alarma** y otro cuando vuelva a **OK**, unos minutos después de que termine stress-ng.

> **Importante:** lanza stress-ng siempre con --timeout. Las instancias t3 funcionan por defecto en modo **ilimitado**: si consumen más CPU que su línea base durante mucho tiempo, se cobra el exceso al presupuesto del laboratorio.

Para ver el estado de tus alarmas desde la CLI, este es el modelo:

```
# AWS CLI
aws cloudwatch describe-alarms --alarm-name-prefix mon-$OWNER \
  --output table --query "MetricAlarms[].{Nombre:AlarmName,
  Estado:StateValue,Metrica:MetricName}"
```

Ejercicios.

- Muestra con describe-alarm-history los cambios de estado de mon-xx-cpu y su hora. Pista: --history-item-type StateUpdate y los campos Timestamp y HistorySummary.
- ¿Cuántos minutos pasaron desde que lanzaste stress-ng hasta el cambio a **En alarma**? Explícalo con el periodo, la regla 3 de 5 y el retraso con que llegan los datos.
- Consulta la métrica CPUCreditBalance de la instancia. ¿Qué le ha pasado durante la prueba?

> **Para pensar:** con 3 de 5, un pico de dos minutos no dispara la alarma. ¿Qué pasaría con 1 de 1? ¿Y con 5 de 5? ¿Qué configuración elegirías para una alarma que despierta a alguien a las tres de la madrugada?
>
> **Para pensar:** si detienes la instancia, la CPU deja de publicarse y, con datos ausentes, la alarma acaba en **Datos insuficientes**. ¿Qué tratamiento de datos ausentes elegirías para una alarma que vigila que un proceso nocturno envía una métrica cada 5 minutos? Pista: que no lleguen datos también es un fallo.

PARTE 6: REINICIO ANTE UN FALLO DE LA INSTANCIA

Ahora el fallo estará **dentro** del sistema operativo: desactivaremos la interfaz de red. El host de AWS sigue sano, así que la comprobación del sistema no detecta nada. La que falla es la de la

instancia, porque EC2 deja de alcanzar el sistema operativo por la red. Aquí no sirve cambiar de host: la solución es **reiniciar**.

Ejercicio. Crea la alarma mon-xx-instancia por el camino que prefieras (desde EC2 o desde CloudWatch), con estos datos:

- Métrica StatusCheckFailed_Instance de mon-xx, con la estadística **Máximo** y un periodo de **1 minuto**.
- Condición: **mayor o igual que 1** en **3 de 3** puntos de datos.
- Acciones: acción de EC2 **Reiniciar esta instancia** y notificación a avisos-xx cuando esté en alarma.

Espera a que la alarma esté en **OK**. Después, conéctate por SSH, averigua el nombre de la interfaz de red (en las t3 suele ser ens5) y desactívala:

```
# mon-xx
ip -br link
sudo ip link set ens5 down
```

La sesión SSH se quedará colgada: la instancia se ha quedado sin red. Ciérrala y sigue el proceso desde la consola:

- En la pestaña **Estado y alarmas** de la instancia, la comprobación de **accesibilidad de la instancia** pasa a fallar, mientras que la del sistema sigue correcta.
- Tras unos 3 a 5 minutos, la alarma pasa a **En alarma**, la instancia se reinicia y recibes el aviso por correo.
- El cambio hecho con ip link no es permanente, así que tras el reinicio la interfaz vuelve a estar activa. Conéctate de nuevo por SSH y comprueba con uptime que la instancia acaba de arrancar.

> **Para pensar:** si el fallo fuera permanente (por ejemplo, un error en /etc/fstab que impide arrancar), ¿se reiniciaría la instancia una y otra vez? Pista: las acciones se ejecutan cuando la alarma **cambia** de estado, no mientras sigue en alarma.
>
> **Para pensar:** ¿por qué no tiene sentido usar **Recuperar** para este fallo, ni **Reiniciar** para un fallo del sistema?

PARTE 7: COMPROBACIÓN DEL ESTADO DE LA APLICACIÓN

Las comprobaciones del sistema y de la instancia dicen si la máquina funciona, pero no si funciona **lo que corre en ella**: una instancia puede aprobar las dos con Apache caído. Para eso EC2 ofrece las **comprobaciones de estado de la aplicación**: AWS se conecta periódicamente a la

aplicación (en nuestro caso, por HTTP al puerto 80) y publica el resultado en la métrica StatusCheckFailed_Application.

Preparación. La comprobación no llega desde tu navegador, sino desde la red de AWS dentro de la VPC, y la regla HTTP de la parte 1 solo admite el rango del instituto. Añade a mon-xx-sg una regla de entrada **HTTP** con origen la red de la VPC: **172.31.0.0/16** en la VPC por defecto. Después, conéctate por SSH y comprueba que Apache responde:

```
# mon-xx
curl -I http://localhost
```

Debe devolver 200 OK. Si tu instancia no tiene Apache, instálalo con sudo apt update && sudo apt install -y apache2.

Crea la comprobación. En la instancia, abre la pestaña **Estado y alarmas**, pulsa el enlace **Administración** junto a «Estado de la aplicación» y después **Crear comprobación de estado de la solicitud**. Usa estos valores:

- **Nombre:** apache-xx.
- **Protocolo:** HTTP, puerto 80, ruta /.
- **Tiempo de espera:** 5 s. **Códigos:** 200. **Umbrales de fallo y de éxito:** 2.
- **Agregación:** Incluido. **Rutas de red:** No especifique las rutas de red (opción predeterminada).

Después:

1. **Asóciala a tu instancia** con el botón **Administrar asociaciones de comprobación de estado**, por ID de instancia.
2. **Espera.** Durante 5 a 10 minutos aparecerá como **Datos insuficientes**. Es normal: mientras tanto, avanza con la parte 8 y vuelve aquí.
3. **Comprueba que pasa a Aprobado** con el motivo ResponseCodeMatched. En la pestaña **Estado y alarmas** verás «Estado de la aplicación: comprobación superada», y su gráfica en **Métricas**.

Provoca el fallo. Para Apache:

```
# mon-xx
sudo systemctl stop apache2
```

En uno o dos minutos, la comprobación pasa a **Error** con el motivo ConnectionRefused y la instancia muestra «3/4 comprobaciones aprobadas». Recarga también la página en el navegador. Después, vuelve a arrancar Apache con sudo systemctl start apache2 y comprueba que la comprobación vuelve a **Aprobado**.

Ejercicio (opcional). Crea la alarma mon-xx-aplicacion. Ve a **CloudWatch > Alarmas > Crear alarma > Seleccionar métrica > EC2 > Métricas por instancia**, busca tu instancia y elige StatusCheckFailed_Application. Verás también una métrica terminada en el ID de tu comprobación (StatusCheckFailed_Application_asc-…), que es la de esa comprobación en concreto: usa la primera. Condición: **Mayor o igual que 1**, en **1 de 1** puntos de datos, con aviso a avisos-xx. Vuelve a parar Apache y comprueba que llega el correo.

> **Si no funciona:** si la comprobación se queda en **Datos insuficientes** o falla con ConnectionTimeout, revisa que esté asociada a la instancia y que mon-xx-sg tenga la regla HTTP desde 172.31.0.0/16.
>
> **Para pensar:** con Apache parado, ¿qué dicen las comprobaciones del sistema y de la instancia? ¿Por qué no basta con ellas para saber si la web funciona?
>
> **Para pensar:** ¿qué diferencia hay entre los motivos ConnectionRefused y ConnectionTimeout? ¿Qué te indica cada uno?

PARTE 8: PANEL DE CLOUDWATCH

Un panel reúne en una pantalla lo que un operador necesita mirar. En **CloudWatch > Paneles > Crear panel**, crea panel-xx con estos widgets y guárdalo:

- **Línea** con CPUUtilization de mon-xx y un periodo de 1 minuto.
- **Estado de la alarma** con las tres alarmas de la práctica.
- **Número** con el máximo de StatusCheckFailed de la instancia.

Ejercicio. Lanza otra vez stress-ng, esta vez durante 5 minutos, y sigue la prueba solo desde el panel. Activa la actualización automática cada minuto.

Abre **Acciones > Ver/editar código fuente**: el panel es un documento JSON. Con él se puede crear el mismo panel en otra cuenta o región con aws cloudwatch put-dashboard.

> **Para pensar:** los tres primeros paneles de la cuenta son gratuitos. ¿Por qué conviene tener pocos paneles bien pensados en lugar de uno por cada recurso?

AMPLIACIÓN: ALARMAS DESDE LA CLI (OPCIONAL)

Si te sobra tiempo, crea una copia de la alarma de CPU con put-metric-alarm. **Ejercicio.** Completa los huecos (___) y ejecuta:

```
# AWS CLI
aws cloudwatch put-metric-alarm --alarm-name mon-$OWNER-cpu-cli \
  --namespace AWS/EC2 --metric-name ___ \
  --dimensions Name=InstanceId,Value=$ID \
  --statistic ___ --period 60 --threshold 70 \
  --comparison-operator ___ \
  --evaluation-periods 5 --datapoints-to-alarm ___ \
  --treat-missing-data missing \
  --alarm-actions $TOPIC --ok-actions $TOPIC
```

Compárala con mon-xx-cpu con describe-alarms y --query: deben coincidir en todo salvo el nombre. Las acciones de EC2 también son ARN: la de reiniciar es arn:aws:automate:us-east-1:ec2:reboot. ¿Cuál será la de recuperar? Compruébalo en la alarma mon-xx-sistema.

PARTE 9: FIN DE LA PRÁCTICA

En la siguiente actividad seguiremos trabajando sobre lo que has montado aquí, así que **no lo borres**. Conserva la instancia mon-xx, el grupo de seguridad mon-xx-sg, el tema avisos-xx con tu suscripción ya confirmada, el panel panel-xx y las tres alarmas.

- Elimina la comprobación de estado apache-xx, que cuesta unos 0,01 USD por hora.
- Borra con aws cloudwatch delete-alarms la alarma mon-xx-aplicacion, si la creaste, y la alarma duplicada mon-xx-cpu-cli, si hiciste la ampliación.
- **Detén** la instancia mon-xx. Detenida no consume horas de cómputo ni envía métricas, así que la monitorización detallada tampoco genera coste. Las alarmas pasarán a **Datos insuficientes**: es lo esperado.
- Pulsa **End Lab**.

> **Importante:** el Learner Lab vuelve a arrancar las instancias detenidas al iniciar cada sesión. Si usas el laboratorio para otra cosa antes de la siguiente actividad, detén mon-xx al empezar. Si no vas a continuar con ella, termínala y borra el resto de los recursos de la práctica.

RESUMEN

Comprobación	Métrica	Qué detecta	Acción habitual
Sistema	`StatusCheckFailed_System`	Fallo del host, la alimentación o la red de AWS.	Recuperar

Comprobación	Métrica	Qué detecta	Acción habitual
Instancia	`StatusCheckFailed_Instance`	Fallo del sistema operativo: red, memoria, kernel o sistema de ficheros.	Reiniciar
EBS adjunto	`StatusCheckFailed_AttachedEBS`	Un volumen EBS de la instancia no responde.	Avisar y revisar
Aplicación	`StatusCheckFailed_Application`	La aplicación no responde o devuelve un código inesperado.	Avisar y revisar el servicio

Necesidad	Comando u opción	Para tener en cuenta
Ver qué métricas hay	`aws cloudwatch list-metrics`	Una métrica aparece unos minutos después de recibir su primer dato.
Leer una métrica	`get-metric-statistics`	Un dato cada 5 min (básica) o cada minuto (detallada).
Monitorización detallada	`aws ec2 monitor-instances`	Tiene coste. unmonitor-instances la desactiva.
Forzar el estado	`set-alarm-state`	Ejecuta las acciones. Dura hasta la siguiente evaluación.
Estado e historial	`describe-alarms / describe-alarm-history`	--alarm-name-prefix filtra por nombre.
Crear o modificar una alarma	`put-metric-alarm`	Si el nombre ya existe, la sustituye.
Avisar por correo	`aws sns publish / tema + suscripción`	La suscripción por correo hay que confirmarla.

Estado de la alarma	Significado
OK	La métrica está dentro del umbral.
En alarma (ALARM)	Se ha cumplido la condición en M de los últimos N periodos. Se ejecutan las acciones al entrar en este estado.
Datos insuficientes	Faltan datos para evaluar: la alarma es nueva, la instancia está detenida o la métrica no llega.